

Solution strategy for the JAX Blinder–Weiss lifecycle solver

Purpose and scope

This note explains the numerical strategy implemented in `blinder_weiss/` and the two Marimo notebooks in `notebooks/`. It is written for an economics graduate student who is familiar with dynamic optimization but may not yet have used direct collocation, nonlinear-programming solvers, or JAX.

The code solves a **deterministic open-loop lifecycle path** for given initial assets and human capital. It does not solve for the full feedback value function $V(A, K, t)$. That distinction is important:

- an HJB method approximates a policy at every point in a state-space grid;
- the present method chooses one optimal state-and-control trajectory from the specified initial condition.

The central numerical idea is to transcribe the continuous-time control problem into one constrained nonlinear program (NLP). JAX differentiates that finite-dimensional NLP automatically, and a standard constrained optimizer solves it. This avoids a high-dimensional value-function grid and avoids the unstable forward/backward iteration of a shooting method. It does **not** make time discretization disappear, so independent ODE integration and mesh convergence remain essential parts of the solution.

1. Economic problem

1.1 States and controls

The economic states are

- $A(t)$: financial assets;
- $K(t) > 0$: human capital.

The original controls are

- $c(t) > 0$: consumption;
- $h(t) \in [0, 1]$: time spent working or training;
- $x(t) \in [0, 1]$: the share of active time allocated to human-capital investment.

Leisure is $1 - h(t)$. When $x = 0$, all active time produces current labor income. When $x = 1$, all active time is used for schooling or training.

1.2 Objective

The implemented objective is

$$\max_{c,h,x} \int_0^T e^{-\rho t} [U(c(t)) + V(1 - h(t))] dt + e^{-\rho T} B(A(T)).$$

The utility components use the power parameterization

$$u(z; p) = \frac{z^p}{p}, \quad p \neq 0.$$

This corresponds to CRRA $\sigma = 1 - p$. The benchmark sets $p = -1$, so $\sigma = 2$, $u(z) = -1/z$, and $u'(z) = z^{-2}$. Separate weights and power parameters are available for consumption, leisure, and the terminal bequest. The bequest is discounted consistently by $e^{-\rho T}$.

1.3 Dynamics

Assets and human capital evolve according to

$$\dot{A}(t) = rA(t) + K(t)h(t)g(x(t)) - c(t),$$

$$\dot{K}(t) = (ax(t)h(t) - \delta) K(t).$$

The earnings–training tradeoff is

$$g(x) = 1.25 - (sx + 0.5)^2, \quad s = \sqrt{1.25} - 0.5.$$

Equivalently,

$$g(x) = 1 - sx - s^2 x^2.$$

It satisfies $g(0) = 1$, $g(1) = 0$, and $g''(x) < 0$. More investment raises human-capital growth but reduces current earnings.

The constraints are

$$A(t) \geq \underline{A}, \quad c(t) > 0, \quad 0 \leq h(t) \leq 1, \quad 0 \leq x(t) \leq 1.$$

The benchmark uses $\underline{A} = 0$. Initial assets and human capital are fixed. Terminal assets and human capital are free, except that terminal assets are kept slightly positive because the benchmark's negative-power bequest is undefined at zero.

1.4 Benchmark calibration

The default numerical benchmark is defined in `model.py`:

Parameter	Value	Interpretation
T	70	Lifecycle horizon
$A(0)$	5	Initial assets
$K(0)$	1	Human-capital normalization
ρ	0.03	Subjective discount rate
r	0.05	Financial return
δ	0.05	Human-capital depreciation
a	0.22	Human-capital productivity
CRRA coefficients	2	Consumption, leisure, and bequest curvature

This is a numerical benchmark, not an empirical estimate reported by Blinder and Weiss. It was selected to generate a well-scaled lifecycle with schooling, on-the-job training, pure work, and retirement. In particular, the legacy value $a = 1$ permits maximum net human-capital growth of $1 - \delta = 0.95$. Over a 70-period horizon that introduces a scale proportional to $e^{0.95 \times 70}$, which is an extreme numerical stress case rather than a reasonable annual calibration.

2. Transformations that remove troublesome boundaries

Two variable transformations are central to the implementation.

2.1 Log human capital

Define

$$y(t) = \log K(t).$$

Then

$$\dot{y}(t) = ax(t)h(t) - \delta.$$

The optimizer therefore works with (A, y) , while reported human capital is recovered as $K = e^y$. This has three benefits:

1. K is positive by construction;
2. multiplicative human-capital growth becomes an additive state equation;
3. large differences in the scale of K are compressed.

The broad numerical bounds $-80 \leq y \leq 80$ are guards against overflow, not economic restrictions expected to bind in the benchmark.

2.2 Training time instead of the investment share

At retirement, $h = 0$, so the original investment share x has no economic meaning. Numerically, trying to recover or optimize x at $h = 0$ creates a division-by-zero boundary. The code instead defines training time

$$q(t) = h(t)x(t).$$

The original control restrictions are exactly equivalent to

$$0 \leq q(t) \leq h(t) \leq 1.$$

The human-capital equation becomes especially simple:

$$\dot{q}(t) = aq(t) - \delta.$$

For positive hours, $x = q/h$. Substituting this into current earnings gives the perspective of g :

$$hg(q/h) = h - sq - s^2 \frac{q^2}{h}.$$

On the feasible cone $0 \leq q \leq h$, the term q^2/h converges to zero as $(h, q) \rightarrow (0, 0)$. The code evaluates the retirement point with an explicit safe branch, so it never divides by zero. The NLP optimizes (c, h, q) directly. The displayed investment share is recovered only after optimization, using $x = 0$ when hours are numerically zero.

This transformation is more than a coding convenience. It gives retirement the unique representation $(h, q) = (0, 0)$, preserves the original feasible set, and avoids clipping an economically meaningful control inside functions that must remain differentiable.

3. From optimal control to a nonlinear program

3.1 Direct transcription

Choose N time intervals of equal length

$$\Delta t = T/N, \quad t_i = i\Delta t, \quad i = 0, \dots, N.$$

At each node the NLP includes

$$z_i = (A_i, y_i), \quad u_i = (c_i, h_i, q_i).$$

The flat decision vector is

$$Z = (z_0, \dots, z_N, u_0, \dots, u_N).$$

There are $5(N + 1)$ decision variables. With the default $N = 24$, the NLP has 125 variables, 50 equality constraints, 25 training-time inequalities, 24 midpoint asset inequalities, and box bounds.

The states are decision variables rather than outputs of a forward simulation. The dynamic equations enter as equality constraints. This is the defining feature of direct transcription and is why it is usually more robust than shooting when terminal conditions, state constraints, or multiple lifecycle regimes are present.

3.2 Hermite–Simpson collocation

Write the transformed dynamics as

$$\dot{z} = f(z, u).$$

For interval $[t_i, t_{i+1}]$, first evaluate

$$f_i = f(z_i, u_i), \quad f_{i+1} = f(z_{i+1}, u_{i+1}).$$

Hermite–Simpson constructs the midpoint state

$$z_{i+1/2} = \frac{z_i + z_{i+1}}{2} + \frac{\Delta t}{8} (f_i - f_{i+1})$$

and uses the linearly interpolated midpoint control

$$u_{i+1/2} = \frac{u_i + u_{i+1}}{2}.$$

The interval defect is

$$d_i = z_{i+1} - z_i - \frac{\Delta t}{6} [f_i + 4f(z_{i+1/2}, u_{i+1/2}) + f_{i+1}].$$

The NLP imposes $d_i = 0$ for every interval. For sufficiently smooth solutions, Hermite–Simpson is a fourth-order accurate collocation method. It usually attains much better accuracy than a first-order time-stepping scheme with relatively few nodes.

The implementation also retains a trapezoid option,

$$z_{i+1} - z_i - \frac{\Delta t}{2}(f_i + f_{i+1}) = 0,$$

mainly for comparison and debugging.

3.3 Objective quadrature

Let the discounted flow payoff be

$$L_i = e^{-\rho t_i} [U(c_i) + V(1 - h_i)].$$

For Hermite–Simpson, the integral is approximated by Simpson quadrature:

$$\int_0^T L(t)dt \approx \sum_{i=0}^{N-1} \frac{\Delta t}{6} (L_i + 4L_{i+1/2} + L_{i+1}).$$

The discounted terminal bequest is then added. Since SciPy minimizes, the code passes the optimizer

$$F(Z) = -\frac{1}{T}J(Z).$$

Multiplication by $-1/T$ changes neither the optimizer nor the economic problem; it simply puts the objective on a more convenient numerical scale.

3.4 Boundary and path constraints

Box bounds impose node-level restrictions on assets, consumption, hours, training time, and log human capital. The nonlinear inequality

$$h_i - q_i \geq 0$$

enforces $q_i \leq h_i$.

Enforcing $A_i \geq 0$ only at nodes is not sufficient for Hermite–Simpson. The cubic state interpolant can dip below zero between two nonnegative endpoints. The implementation therefore also imposes

$$A_{i+1/2} \geq \underline{A}$$

at every collocation midpoint. This detail is essential when the optimal path contains a borrowing-limit arc. Without it, a solver may report a feasible NLP whose implied continuous asset path violates the economic state constraint between nodes.

The path constraint is still enforced at finitely many collocation points, not at every real-valued time. The independent dense integration and mesh study described below quantify the remaining between-point error.

3.5 Scaling

Constrained optimizers compare derivatives and residuals across equations. If one state is naturally of order 10^5 and another is of order one, an unscaled residual norm can be misleading and the quadratic subproblem can be ill-conditioned.

The code divides asset defects by a benchmark asset scale and leaves the log-human-capital defect on its natural order-one scale. It also divides the objective by T . These transformations do not alter the feasible set or optimizer; they improve the conditioning of the numerical problem.

3.6 Initial guess and coarse-to-fine warm starts

The initial guess uses constant assets, fixed hours, and a small fixed investment share. Log human capital follows its implied linear path, and consumption is chosen node by node so that asset growth is zero. This guess is positive and exactly feasible for the trapezoid dynamics. It is also a useful starting point for Hermite–Simpson even though it need not satisfy those higher-order defects exactly.

For a mesh sequence, a converged coarse solution is linearly interpolated onto the next mesh. Warm starts help the optimizer remain near the same local solution and reduce the work needed on finer meshes. They do not establish global optimality.

4. What JAX contributes

4.1 Automatic differentiation

The discretized objective and constraints are compositions of elementary operations: addition, multiplication, powers, exponentials, reshaping, and stacking. JAX records this computational graph and applies the chain rule to it. The implementation uses

- `jax.grad` for the scalar objective gradient $\nabla F(Z)$;
- `jax.jacrev` for constraint Jacobians;
- `jax.vmap` to evaluate the same dynamic equation at all time nodes;
- `jax.jit` to compile repeatedly evaluated functions.

Automatic differentiation is different from symbolic differentiation and from finite differences. It evaluates derivatives of the actual implemented function to floating-point accuracy, without selecting a perturbation size. This removes the cancellation and step-size tradeoffs of finite-difference Jacobians.

However, JAX differentiates the **discretized NLP**, not the unknown exact continuous-time solution. A perfectly accurate gradient cannot repair an inadequate time mesh or an omitted path constraint.

4.2 Reverse-mode differentiation

For a scalar objective with many decision variables, reverse-mode automatic differentiation computes the full gradient at a cost comparable to a small multiple of evaluating the objective. `jax.grad` is therefore well suited to this problem. `jax.jacrev` applies the same reverse-mode machinery to each constraint output to form the Jacobian required by SciPy.

4.3 Compilation and the NumPy bridge

The first evaluation of a jitted function includes compilation. Subsequent evaluations reuse machine code specialized to the array shapes and closed-over parameters. The solver deliberately triggers compilation before entering SciPy so compilation time is not mistaken for a slow optimizer iteration.

SciPy calls ordinary Python functions with NumPy arrays. Thin wrappers convert those arrays to JAX arrays, call the compiled JAX function, and convert the result back to NumPy. Thus JAX supplies the derivatives while SciPy or IPOPT manages the nonlinear-programming algorithm.

4.4 Why 64-bit arithmetic is enabled

JAX often defaults to 32-bit floating-point arithmetic. A relative precision near 10^{-7} is not enough when an optimizer is asked to distinguish constraint residuals around 10^{-9} over a 70-period horizon. The package enables `jax_enable_x64` before constructing model arrays. The resulting double-precision arithmetic is important for feasibility, multiplier, and KKT diagnostics.

4.5 Differentiability choices in the model

The optimizer-facing functions avoid clipping controls internally. Clipping would insert nondifferentiable kinks and could make the derivative inconsistent with the explicit NLP bounds. Bounds are handled by the optimizer instead.

The safe retirement evaluation of $hg(q/h)$ is written analytically in terms of (h, q) . It is continuous on the feasible cone and gives JAX a well-defined computation at $(0, 0)$.

5. Nonlinear optimization theory

5.1 Generic NLP form

After transcription, the problem has the form

$$\min_Z F(Z)$$

subject to

$$C(Z) = 0, \quad G(Z) \geq 0, \quad \ell \leq Z \leq u.$$

Here C contains the initial conditions and collocation defects. G contains the training-time and midpoint-asset slacks. The box bounds contain the remaining pointwise restrictions.

This NLP is smooth but generally nonconvex. Concavity of period utility alone does not imply that every transformed dynamic constraint or the complete finite-dimensional feasible set is convex. A solver’s success therefore means that it found a locally optimal feasible point satisfying first-order conditions to tolerance, not that it proved a unique global optimum.

5.2 Sequential quadratic programming and SLSQP

The default optimizer is SLSQP, a sequential quadratic programming method. At an iterate Z_k , SQP approximately:

1. forms a quadratic model of the Lagrangian;
2. linearizes the equality and active inequality constraints;
3. solves the resulting constrained quadratic subproblem for a step;
4. uses a line search and merit function to balance objective improvement and feasibility;
5. updates a quasi-Newton approximation to the Lagrangian Hessian.

JAX supplies the objective gradient and constraint Jacobians. SLSQP builds the second-order approximation internally, so the code does not provide a Hessian.

The borrowing limit can become active only after many iterations. At that point SLSQP’s accumulated quasi-Newton approximation may describe the previous active set better than the final one. The implementation therefore performs one warm restart from the reported solution. Restarting resets the quasi-Newton model while preserving the candidate path. The restarted result is retained only if the optimizer again succeeds and the minimized objective does not worsen.

5.3 Other backends

The solver also exposes:

- `trust-constr`, SciPy’s trust-region constrained method;
- `ipopt`, an interior-point method accessed through `cyipopt`.

IPOPT is designed for large sparse NLPs. The code passes conservative local sparsity patterns for dynamic and path constraints. The current JAX wrapper

still constructs a dense Jacobian before extracting those sparse entries, so very large meshes would benefit from a future block-local Jacobian implementation. At the benchmark sizes, the default dense SLSQP route is simple and reliable.

5.4 KKT conditions

For the minimization problem, use the sign convention

$$\mathcal{L}(Z, \lambda, \mu) = F(Z) - \lambda' C(Z) - \mu' G(Z), \quad \mu \geq 0.$$

At a regular local solution, the Karush–Kuhn–Tucker conditions include:

Primal feasibility

$$C(Z) = 0, \quad G(Z) \geq 0, \quad \ell \leq Z \leq u.$$

Dual feasibility

$$\mu \geq 0,$$

together with correctly signed multipliers on active box bounds.

Complementary slackness

$$\mu_j G_j(Z) = 0.$$

Stationarity

$$\nabla_Z \mathcal{L} = 0$$

for interior variables, with one-sided conditions at bounds.

For example, at a lower bound a positive Lagrangian derivative is consistent with optimality: moving in the only feasible direction would increase the minimized objective. A negative derivative is a violation because increasing the variable would be an improving feasible direction. The diagnostic therefore projects the Lagrangian gradient as follows:

- interior: retain the derivative;
- lower bound: retain only its negative part;
- upper bound: retain only its positive part.

The maximum absolute projected component is reported as the projected KKT residual. For SLSQP, the code uses the signed equality and inequality multipliers returned by SciPy. Other backends fall back to a constrained least-squares reconstruction from active-constraint Jacobians.

This residual is calculated for the scaled NLP actually sent to the optimizer. Its numerical threshold should therefore be interpreted together with the scaling conventions described above, not as an unscaled welfare derivative.

A small dynamic defect is not a substitute for this stationarity check. A candidate can be nearly feasible yet still admit a large feasible improvement in utility.

6. Validation beyond optimizer success

The implementation intentionally distinguishes `optimizer_success` from an economically and numerically accepted path.

6.1 Feasibility checks

The solver recomputes

- the scaled initial-condition and dynamic residuals;
- the minimum value of $h - q$;
- the minimum midpoint asset slack.

The remaining pointwise restrictions are supplied as explicit optimizer box bounds. The result object and diagnostics report the economically important minima, including assets, consumption, and leisure.

The default maximum constraint tolerance is 10^{-7} .

6.2 Projected KKT check

The diagnostic requires a projected KKT residual no larger than 10^{-4} . This is deliberately separate from the optimizer’s own success flag because termination criteria based on objective change can occasionally declare success before stationarity is satisfactory.

6.3 Independent Diffrax integration

Collocation feasibility only says that the chosen polynomial representation satisfies the discretized equations. As an independent check, the code:

1. linearly interpolates the optimized controls (c, h, q) between nodes;
2. integrates the original transformed ODE with Diffrax’s adaptive Tsitouras 5/4 Runge–Kutta solver;
3. saves the integrated path on a grid ten times denser than the collocation mesh;
4. compares the independently integrated states with the NLP states at the original nodes;
5. checks the minimum asset value over the denser grid.

With independent integration enabled, acceptance requires

$$\max_i |A_i^{\text{ODE}} - A_i^{\text{NLP}}| \leq 10^{-3} \max(1, \max_i |A_i|),$$

$$\max_i \frac{|K_i^{\text{ODE}} - K_i^{\text{NLP}}|}{\max(1, K_i^{\text{NLP}})} \leq 10^{-4},$$

and the dense integrated asset path may fall below the borrowing limit by no more than 10^{-4} times the asset scale. This last allowance recognizes that a finite collocation mesh approximates a continuous boundary arc; the violation should shrink under refinement.

6.4 Mesh convergence

The convergence notebook solves a sequence such as

$$N \in \{8, 12, 24\}$$

and compares utility, controls, KKT residuals, and independent integration errors. A convincing result should show:

- a stable lifecycle regime sequence;
- a stabilizing objective;
- declining independent ODE error;
- declining between-node borrowing-limit error;
- continued feasibility and stationarity.

In the current environment, the benchmark sequence gives approximately:

Intervals	Utility	KKT residual	Asset ODE error	Dense minimum assets	Accepted
8	- 99.616399	7.07×10^{-7}	0.06145	-0.23726	No
12	- 99.540474	1.23×10^{-8}	0.02895	-0.06082	No
24	- 99.494016	2.37×10^{-6}	0.00219	-6.64×10^{-5}	Yes

The coarse solutions satisfy their NLP constraints but fail the independent continuous-path test. That is useful information rather than a solver failure: it demonstrates why collocation residuals alone are not enough.

6.5 Continuous-time first-order conditions

The diagnostic module also reconstructs current-value costates and evaluates the familiar Pontryagin conditions. For the Hamiltonian

$$H = U(c) + V(1 - h) + \mu_A[rA + Khg(x) - c] + \mu_K[(axh - \delta)K],$$

the interior control conditions include

$$U'(c) = \mu_A,$$

$$-V'(1 - h) + \mu_A Kg(x) + \mu_K axK = 0,$$

$$hK[\mu_A g'(x) + a\mu_K] = 0.$$

At control bounds these become one-sided inequalities. They are useful economic diagnostics, but the asset state constraint adds a multiplier to the costate system while $A = \underline{A}$. The simple reconstructed costate does not include that state-constraint multiplier. The code therefore excludes the portion of the path at or before the last borrowing-limit contact from these continuous-time residuals. The full finite-dimensional KKT diagnostic remains the primary stationarity check because it includes the active path constraint.

6.6 Lifecycle regimes

For presentation, nodes are classified as:

- **schooling:** x is numerically one;
- **on-the-job training:** $0 < x < 1$ with positive hours;
- **pure work:** x is numerically zero with positive hours;
- **retirement:** hours are numerically zero.

The default accepted path follows

$$\text{schooling} \rightarrow \text{on-the-job training} \rightarrow \text{pure work} \rightarrow \text{retirement}.$$

This ordering is an economic plausibility check, not a mathematical constraint imposed on the optimizer.

7. Reading the code

`blinder_weiss/model.py`

This file contains the parameter tuple, utility functions, the earnings tradeoff, the (h, q) earnings perspective, and the transformed dynamics. These functions are written with `jax.numpy` so JAX can trace and differentiate them.

blinder_weiss/transcription.py

This file defines the flat decision-vector layout, time grid, Hermite–Simpson midpoint, dynamic defects, quadrature objective, path slacks, bounds, initial guess, and coarse-to-fine interpolation.

blinder_weiss/solver.py

This file builds jitted objective and constraint functions, obtains JAX derivatives, wraps them for SciPy, dispatches to SLSQP, trust-constr, or IPOPT, performs the SLSQP restart, and converts the optimized vector into economically named paths.

blinder_weiss/diagnostics.py

This file computes projected KKT residuals, continuous-time FOC checks, DiffraX integration errors, borrowing-limit diagnostics, and regime labels. It determines whether a one-mesh result passes the implemented acceptance thresholds.

notebooks/01_baseline.py

The baseline Marimo notebook exposes the main calibration and numerical choices, solves one path after the run button is pressed, displays acceptance metrics, and plots consumption, work, training, human capital, assets, and the potential wage.

notebooks/02_mesh_convergence.py

The convergence notebook solves increasing meshes with warm starts and shows which meshes pass the independent continuous-path validation.

tests/

The tests verify economic primitives, the control transformation, autodiff dtypes and finiteness, initial-guess feasibility, numerical solution feasibility, midpoint borrowing-limit enforcement, KKT quality, and the four lifecycle regimes.

8. Reproducing the benchmark

Enter the project environment and synchronize dependencies:

```
devenv shell
uv sync
```

Open the baseline notebook:

```
uv run marimo edit code/jax/notebooks/01_baseline.py
```

Open the convergence notebook:

```
uv run marimo edit code/jax/notebooks/02_mesh_convergence.py
```

The reusable Python API can also be called directly:

```
from blinder_weiss import diagnose_path, solve_lifecycle

result = solve_lifecycle()
diagnostics = diagnose_path(result)

print(diagnostics.accepted_success)
print(diagnostics.regime_sequence)
print(diagnostics.projected_kkt_residual)
```

On the current environment, the standalone 24-interval benchmark has utility about -99.494015 , maximum constraint violation about 1.9×10^{-11} , projected KKT residual about 3.1×10^{-6} , and the four regimes in the expected order. Small last-digit differences across BLAS implementations, CPU architectures, and package versions are normal.

Run the automated checks with:

```
uv run pytest
uv run ruff check .
uv run basedpyright code/jax/blinder_weiss tests
uv run marimo check --strict \
  code/jax/notebooks/01_baseline.py \
  code/jax/notebooks/02_mesh_convergence.py
```

9. How to interpret and extend the result

What an accepted result establishes

An accepted result establishes that, for the chosen mesh and starting point:

- the optimizer reported success;
- the discretized dynamics and constraints are feasible to tight tolerance;
- the NLP is approximately stationary according to its KKT conditions;
- a separate ODE solver closely reproduces the states;
- the dense validation path respects the borrowing limit to the stated numerical tolerance.

The mesh notebook adds evidence that these conclusions are not artifacts of a particular time grid.

What it does not establish

An accepted result does not prove:

- global optimality of the nonconvex NLP;
- exact feasibility at every continuous time;

- that the benchmark is the paper’s empirical calibration;
- the full feedback policy for arbitrary (A, K, t) ;
- differentiability of the optimized solution with respect to parameters.

JAX differentiates the objective and constraints with respect to the NLP variables. The present code does not differentiate *through the optimizer*. Parameter sensitivities should initially be studied by resolving the problem under perturbations or continuation, not by assuming that `jax.grad` can be applied to `solve_lifecycle`.

Recommended research workflow

1. **Lock down the economic specification.** Verify timing, discounting, terminal conditions, units, and the intended parameterization against the paper or replication target.
2. **Solve the documented benchmark.** Confirm the expected regimes and all acceptance diagnostics.
3. **Run mesh convergence.** Do not treat a coarse feasible NLP as a continuous solution.
4. **Use multiple starts.** Perturb the initial control path or use parameter continuation to look for competing local optima.
5. **Compare objective levels, not only plots.** Similar-looking paths can have meaningfully different welfare, and different paths can be nearly tied.
6. **Perform comparative statics gradually.** Warm-start nearby parameter values and re-run all diagnostics at each value.
7. **Increase numerical sophistication only when needed.** Very fine meshes may justify block-sparse JAX Jacobians, IPOPT tuning, adaptive mesh refinement, or multiple shooting.

If the research question eventually requires policy rules for many initial states or aggregate uncertainty, direct collocation of one path is no longer the final object. It can still serve as a high-quality benchmark for an HJB, dynamic-programming, or heterogeneous-agent implementation.